



FIXMATE

Local Service Booking Platform

PRODUCT REQUIREMENTS DOCUMENT

Flutter

Node.js

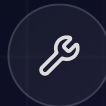
Express.js

MongoDB

JWT



Customer



Service Provider



Booking



Table of Contents

	01	Product Overview	3		13	Database Design	17
	02	Problem Statement	4		14	Backend Architecture	18
	03	Goals & Objectives	5		15	Authentication & Authorization	20
	04	User Roles	6		16	Validation & Error Handling	21
	05	User Flow	7		17	Security Requirements	22
	06	Core Features	8		18	MVP Scope	23
	07	Service Management	9		19	Future Enhancements	24
	08	Booking System	10		20	Testing Strategy	25
	09	Provider Availability	12		21	Development Phases	26
	10	Reviews & Ratings	13		22	Final Deliverables	27
	11	Admin System	14		23	Portfolio Presentation	28
	12	API Requirements	15				



This document specifies the product, data model, and API surface for FixMate's MVP release. It is written for engineers, designers, and reviewers evaluating scope ahead of implementation.



Product Overview

What FixMate is, who it serves, and the stack it runs on.

FixMate is a full-stack local service booking platform that connects everyday customers with trusted local professionals — plumbers, electricians, cleaners, technicians, and more. Customers discover and book services in a few taps; providers manage their own profile, services, and schedule; administrators keep the marketplace healthy from a single dashboard.



Purpose

Connect customers with trusted local service providers through a simple, reliable booking platform — replacing phone calls and word-of-mouth with a structured flow.



Main Users

Three roles share the platform: the **Customer** booking services, the **Service Provider** fulfilling them, and the **Admin** overseeing the marketplace.



Main Platform

A Flutter mobile application backed by a Node.js + Express.js REST API, with MongoDB and Mongoose handling persistence.

3

User roles

4

Core collections

5

Booking states

REST

API architecture

Platform at a glance

Customer journey, simplified



Discover
nearby services



Compare &
book a provider



Track the
booking status



Rate the
completed job



Problem Statement

The gaps FixMate is designed to close in the local services market.

Finding and booking a reliable local professional is still largely an offline, informal process. That creates friction for customers and lost demand for providers. FixMate addresses six recurring problems:



Hard to find providers

Customers rely on word-of-mouth or unverified listings to find a plumber, electrician, or cleaner nearby.



No centralized information

Pricing, availability, and service details live across calls, flyers, and social posts — never in one place.



Manual booking process

Booking still means phone tag and back-and-forth messages to confirm a date, time, and job scope.



No way to track requests

Once a request is sent, customers have no visibility into whether it was seen, accepted, or scheduled.



Reviews aren't trustworthy

Feedback is scattered or unverifiable, making it hard to judge a provider's real track record.



Poor communication

Customers and providers lack a shared channel for status updates, leading to missed appointments.



The compounding effect: each gap on its own is a minor inconvenience, but together they push customers back toward informal channels and leave good providers under-booked. FixMate replaces all six with one structured, trackable flow.



Goals & Objectives

What the MVP release needs to achieve for each side of the marketplace.

Six measurable objectives guide scope decisions for the first release. Every feature in this document maps back to one of these goals.



Goal 01 — Simple discovery

Let customers find the right nearby service in a few taps, with clear categories and search.



Goal 02 — Easy booking

Reduce booking a provider to a short, guided flow — no calls, no back-and-forth.



Goal 03 — Availability control

Give providers a simple way to set working days and hours so they're never over-booked.



Goal 04 — Transparent status

Keep both sides in sync with a single source of truth for where a booking stands.



Goal 05 — Trustworthy reviews

Let only verified, completed bookings generate a review — no fake or duplicate feedback.



Goal 06 — Admin oversight

Give administrators the tools to manage users, providers, and content without touching the database.

How goals map to the data model

Discovery and booking goals are served by the **services** and **bookings** collections; trust and oversight goals are served by **reviews** and role-based access across all four collections. See *Section 13 — Database Design* for the full schema.



User Roles

Three roles, each with a distinct set of permissions across the platform.



Customer

Discovers & books services

- ✓ Register / login
- ✓ Browse services
- ✓ Search services
- ✓ Filter services
- ✓ View providers
- ✓ Book services
- ✓ Track booking status
- ✓ Cancel eligible bookings
- ✓ View booking history
- ✓ Review completed services



Service Provider

Fulfills booked services

- ✓ Register / login
- ✓ Manage profile
- ✓ Manage services
- ✓ Set availability
- ✓ View bookings
- ✓ Accept / reject bookings
- ✓ Update booking status
- ✓ View booking history
- ✓ Receive customer reviews



Admin

Oversees the marketplace

- ✓ Manage users
- ✓ Manage providers
- ✓ Manage services
- ✓ Manage bookings
- ✓ Manage reviews
- ✓ Activate / deactivate users
- ✓ Delete inappropriate reviews



Every API route checks the requester's `role` claim from the JWT before executing — a customer token can never call a provider- or admin-only endpoint, even with a valid, unexpired token.



User Flow

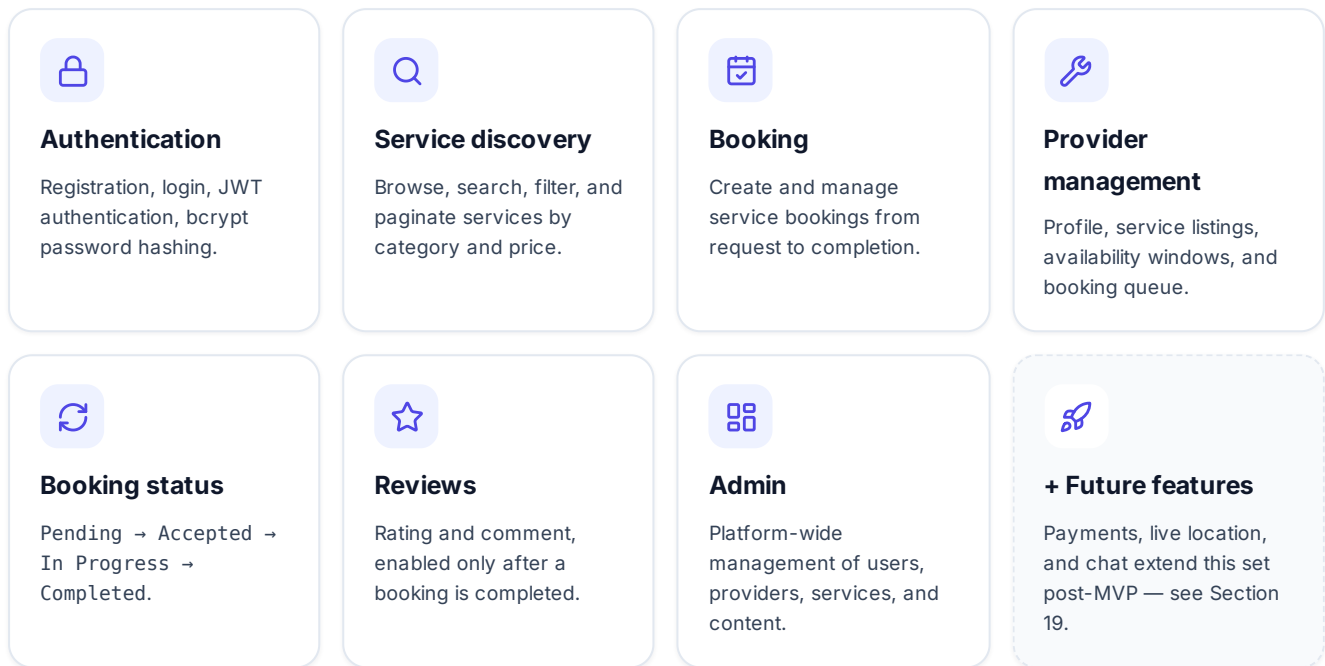
The end-to-end path from opening the app to leaving a review.

- 1 Customer opens FixMate**
Entry point for the whole journey
- 2 Register / Login**
JWT-based authentication
- 3 Browse services**
Paginated catalog by category
- 4 Search / filter**
By name, category, price, availability
- 5 Select provider**
From matching results
- 6 View provider profile**
Services offered, rating, availability
- 7 Select service**
Choose the specific job to book
- 8 Choose date & time**
Within the provider's availability window
- 9 Enter address**
Service location and any notes
- 10 Create booking**
Booking is created with status PENDING
- 11 Provider receives request**
Appears in the provider's booking queue
- 12 Provider accepts / rejects**
Status moves to ACCEPTED or REJECTED
- 13 Service started**
Provider marks work IN_PROGRESS
- 14 Service completed**
Provider marks the booking COMPLETED
- 15 Customer reviews provider**
Rating + comment, tied to this booking

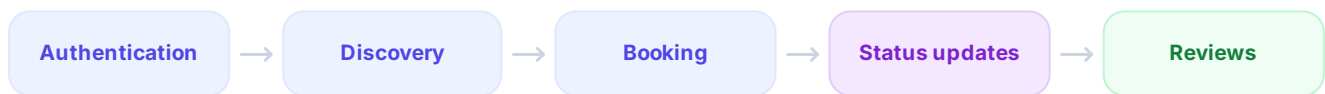


Core Features

The seven feature groups that make up the FixMate MVP.



How the features connect



Admin capabilities run alongside every stage of this chain rather than at a single point in it.



Service Management

The service catalog schema, its API surface, and how customers narrow it down.

Every bookable offering is a **service** document owned by exactly one provider. Providers create and update their own services; customers only ever see active ones.

Service fields

Field	Notes
<code>name</code>	Service name shown to customers
<code>description</code>	Free text detail of the job
<code>category</code>	Plumbing, electrical, cleaning, etc.
<code>price</code>	Base price for the service
<code>provider</code>	Reference to the owning user
<code>isAvailable</code>	Provider can pause bookability
<code>isActive</code>	Soft-delete / visibility flag
<code>createdAt</code>	Set on creation
<code>updatedAt</code>	Refreshed on every edit

Search & filter

Customers narrow the catalog with any combination of:

 Search by name

 Search by category

 Filter by price

 Filter by availability

 Pagination

 Sorting

 Results are always scoped to `isActive: true` — deactivated services never reach the discovery endpoints.

Service API endpoints

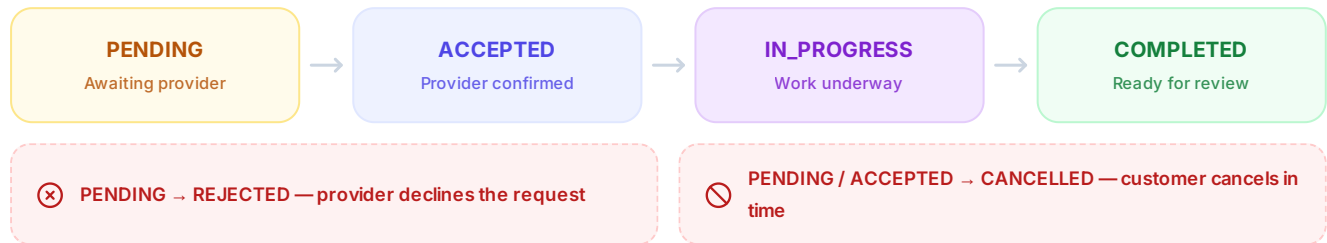
Method	Endpoint	Purpose	Access
POST	<code>/api/services</code>	Create a new service	Provider
GET	<code>/api/services</code>	List / search / filter services	Public
GET	<code>/api/services/:id</code>	Get a single service	Public
PUT	<code>/api/services/:id</code>	Update a service	Owning provider
DELETE	<code>/api/services/:id</code>	Deactivate a service	Owning provider / Admin



Booking System

The booking lifecycle, its states, and the data captured at each step.

Booking status flow



Booking information

Field	Notes
<code>customer</code>	Reference to the booking customer
<code>provider</code>	Reference to the assigned provider
<code>service</code>	Reference to the booked service
<code>bookingDate</code>	Requested calendar date
<code>bookingTime</code>	Requested time slot

Field	Notes
<code>address</code>	Service location text
<code>notes</code>	Optional customer notes
<code>price</code>	Snapshot of the service price
<code>status</code>	One of the five states above
<code>createdAt</code> / <code>updatedAt</code>	Lifecycle timestamps



Booking APIs

Endpoints and who is allowed to call each one.

Method	Endpoint	Purpose	Access
POST	/api/bookings	Create a booking	Customer
GET	/api/bookings	List bookings (own bookings only)	Customer / Provider
GET	/api/bookings/:id	Get a single booking	Owner / assigned provider
PATCH	/api/bookings/:id/status	Advance the booking status	Provider
PATCH	/api/bookings/:id/cancel	Cancel an eligible booking	Customer

Customer can

- Create a new booking request
- View their own bookings and history
- Cancel a booking while still eligible

Provider can

- View bookings made against their services
- Accept or reject a pending request
- Advance status through to completed



Status transitions are validated server-side — a booking can only move forward along the states in Section 08, never skip or reverse a step, and only the assigned provider can trigger a transition.



Provider Availability

How providers control when they can be booked.

Each provider configures a simple weekly availability window. Bookings can only be created against days and times that fall inside it.



Monday — Friday

09:00 AM → 06:00 PM

✓ Available



isAvailable

Master on / off switch for accepting new bookings.



workingDays

Which days of the week the provider operates.



startTime

Earliest bookable time each working day.



endTime

Latest bookable time each working day.



The booking API re-validates the requested date and time against the provider's current availability at creation time — a stale client can never create a booking outside the provider's window.



Reviews & Ratings

How feedback is captured and rolled up into a trustworthy provider score.



4.8 / 5.0

Average provider rating — recalculated on every new review

Review rules

- ✓ Rating is an integer from 1 to 5
- ✓ Only customers can submit a review
- ✓ The related booking must be COMPLETED
- ✓ Exactly one review per booking
- ✓ The reviewer must own the booking
- ✓ Provider average rating recalculates on write

Review document

customer	provider	booking	rating
ObjectId	ObjectId	ObjectId	Number (1–5)
comment			
String, optional			



Admin System

A single dashboard for keeping the marketplace healthy.

The admin role has platform-wide read and moderation access across every collection, exposed through five dashboard areas:



Users

View accounts, activate or deactivate customers and providers.



Providers

Oversee provider profiles and their published service catalog.



Services

Review and, if needed, deactivate listings platform-wide.



Bookings

Full visibility into booking volume and status across the platform.



Reviews

Remove reviews that violate content or authenticity rules.



Guardrail

Every admin action is still logged and scoped — admins moderate, they don't impersonate.

Admin can

- ✓ Manage users, providers, and services
- ✓ Activate / deactivate any user account
- ✓ Manage bookings and reviews platform-wide
- ✓ Delete inappropriate reviews



API Requirements

The complete REST surface — one reference table across every resource.

Group	Method	Endpoint	Purpose
Auth	POST	/api/auth/register	Create an account (customer or provider)
Auth	POST	/api/auth/login	Authenticate and receive a JWT
Services	POST	/api/services	Create a service (provider)
Services	GET	/api/services	List / search / filter services
Services	GET	/api/services/:id	Get a single service
Services	PUT	/api/services/:id	Update a service
Services	DELETE	/api/services/:id	Deactivate a service
Bookings	POST	/api/bookings	Create a booking
Bookings	GET	/api/bookings	List own bookings
Bookings	GET	/api/bookings/:id	Get a single booking
Bookings	PATCH	/api/bookings/:id/status	Advance booking status
Bookings	PATCH	/api/bookings/:id/cancel	Cancel an eligible booking
Reviews	POST	/api/reviews	Submit a review for a completed booking
Reviews	GET	/api/providers/:id/reviews	List a provider's reviews
Reviews	DELETE	/api/reviews/:id	Remove an inappropriate review (admin)
Admin	GET	/api/admin/users	List all users
Admin	PATCH	/api/admin/users/:id/status	Activate / deactivate a user
Admin	GET	/api/admin/bookings	List all bookings platform-wide

Full field-level detail for Services and Bookings appears in Sections 07 and 08. This table is the single source of truth for the API surface as a whole.



API Response Format

Every endpoint replies with the same envelope, success or failure.

SUCCESS RESPONSE

2xx

ERROR RESPONSE

4xx / 5xx

```
{
  "success": true,
  "message": "Operation successful",
  "data": {}
}
```

```
{
  "success": false,
  "message": "Something went wrong",
  "errors": []
}
```

Common status codes

Code	Meaning	Used when
200	OK	Request succeeded (read or update)
201	Created	A new resource was created
400	Bad Request	Validation failed on the request body
401	Unauthorized	Missing or invalid JWT
403	Forbidden	Valid token, wrong role for this route
404	Not Found	Resource / ObjectId does not exist
500	Server Error	Unexpected failure, caught by the global handler



Database Design

Four MongoDB collections and the references that connect them.

👤 users	
name	String
email	String, unique
password	String, hashed
role	Enum
isActive	Boolean

🏠 services	
name	String
description	String
category	String
price	Number
provider	ObjectId → users
isActive	Boolean

📅 bookings	
customer	ObjectId → users
provider	ObjectId → users
service	ObjectId → services
bookingDate	Date
bookingTime	String
address	String
notes	String
price	Number
status	Enum

★ reviews	
customer	ObjectId → users
provider	ObjectId → users
booking	ObjectId → bookings
rating	Number (1–5)
comment	String

Relationships

🔗 services.provider → users

🔗 bookings.customer → users

🔗 bookings.provider → users

🔗 bookings.service → services

🔗 reviews.booking → bookings

🔗 reviews.customer / .provider → users



Backend Architecture

How a request travels from the mobile app down to the database.



Supporting layers — applied on every request

 JWT Authentication

 Validation

 Authorization

 Error Handling

Controllers stay thin — they parse the request and format the response. Business logic lives in the service layer, which is the only layer that talks to Mongoose models. This keeps routes testable and keeps MongoDB access in one place.



Project Structure

Repository layout for the Node.js backend.

```
fixmate-backend/
|
├─ src/
|   ├─ config/
|   ├─ controllers/
|   ├─ middleware/
|   ├─ models/
|   ├─ routes/
|   ├─ services/
|   ├─ utils/
|   └─ validators/
|   ├─ app.js
|   └─ server.js
|
├─ .env
├─ .env.example
├─ .gitignore
├─ package.json
└─ README.md
```

Why this layout

Each concern gets its own folder so a new contributor can predict where code lives: routes wire URLs to controllers, controllers call services, services call models.

Environment config

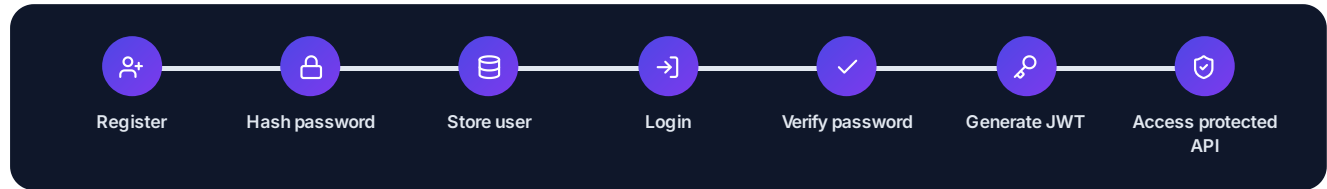
.env holds secrets (Mongo URI, JWT secret) and is git-ignored; .env.example documents the required keys with no real values.



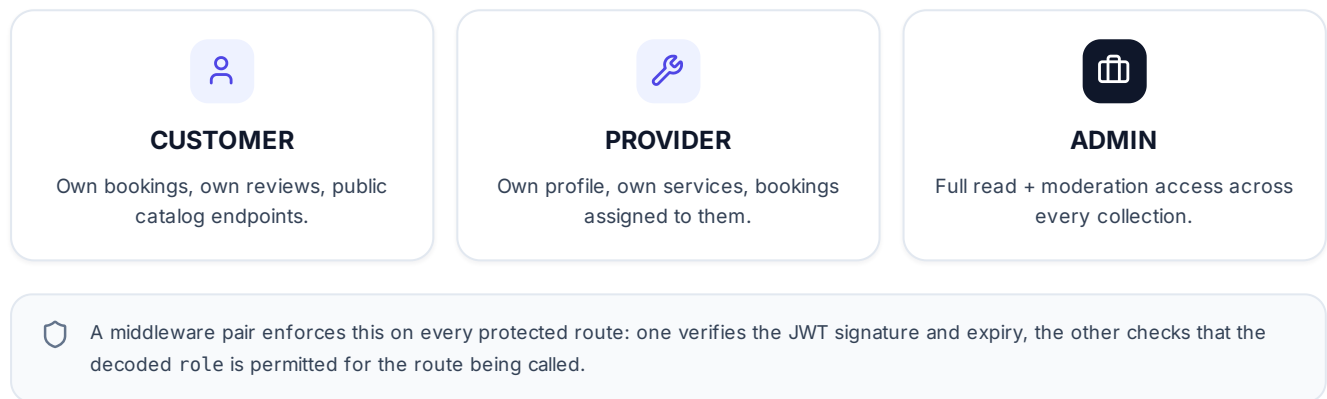
Authentication & Authorization

How identity is established, and how access is restricted once it is.

Authentication flow — JWT + bcrypt



Authorization — role-based access





Validation & Error Handling

What gets checked before a request reaches business logic, and what happens when it fails.



What's validated on every request

- ✓ Required fields present on the body
- ✓ Email format validation
- ✓ Password strength validation
- ✓ MongoDB ObjectId format validation
- ✓ Invalid booking status transitions blocked
- ✓ Unauthorized access prevented pre-controller
- ✓ Global error handler catches all thrown errors
- ✓ Proper HTTP status code on every response
- ✓ Consistent error envelope, never a raw stack trace



A single Express error-handling middleware sits at the end of the chain. Every thrown or forwarded error passes through it, so response shape and status codes stay consistent across all routes — no controller formats its own error response.



Security Requirements

Baseline protections every environment must ship with.



Password hashing

Every password is hashed with bcrypt before it touches the database.



JWT authentication

Signed, expiring tokens protect every non-public route.



Role-based authorization

Every route enforces which roles are permitted to call it.



Environment variables

Secrets live in .env, never committed or hard-coded.



CORS configuration

Only approved origins can call the API from a browser context.



Input validation

Every request body is validated before it reaches business logic.



Safe error messages

Errors describe the problem without leaking internals or stack traces.



No password exposure

The password field is excluded from every user query and response.



No sensitive data leaks

API responses are shaped to return only what the caller is owed.



MVP Scope

Everything required for a working first release — nothing more.

✓ In scope for MVP

- | | |
|-------------------------------------|---------------------------------|
| ✓ Authentication (register / login) | ✓ Booking creation & management |
| ✓ JWT-based sessions | ✓ Booking status lifecycle |
| ✓ Customer role | ✓ Reviews & ratings |
| ✓ Provider role | ✓ MongoDB persistence |
| ✓ Admin role | ✓ Request validation |
| ✓ Service listings | ✓ Global error handling |
| ✓ Search & filter | ✓ Baseline security hardening |
| ✓ Provider availability | ✓ Postman test collection |

- 🎯 Scope is deliberately narrow: a customer can find, book, and review a provider, and a provider can manage their side of that same loop — end to end, with nothing half-built.



Future Enhancements

Deliberately out of scope for v1 — candidates for the next iteration.



Planned for later



Push notifications



Online payments



Google Maps



Live location



In-app chat



Advanced analytics



Provider verification



Coupons



Subscription plans



None of these block the MVP. Each is additive to the existing data model — for example, payments extend the booking document rather than replace it.



Testing Strategy

How the API surface gets verified before release.



API testing — Postman

A maintained Postman collection covers every endpoint in Section 12, organized folder-by-folder to match the resource groups below.

Test categories

- ✓ Authentication — register, login, token expiry
- ✓ Authorization — role checks on protected routes
- ✓ Services — CRUD, search, filter, pagination
- ✓ Bookings — full status lifecycle, cancellation
- ✓ Reviews — one-per-booking, ownership checks
- ✓ Admin — moderation and account status endpoints
- ✓ Validation — malformed and missing-field payloads
- ✓ Error handling — correct status code per failure



Every endpoint is tested for at least one success case and one expected-failure case — an unauthenticated call, an invalid payload, or a permission violation — before it's considered done.



Development Phases

A suggested build order, from empty repo to GitHub-ready project.

1

Project Setup

Repo, structure, environment config

2

Authentication

Register, login, JWT, bcrypt

3

Authorization

Role-based route protection

4

Services

CRUD, search, filter, pagination

5

Provider Availability

Working days & hours

6

Booking System

Lifecycle & status transitions

7

Reviews & Ratings

Submission rules & averages

8

Admin System

Moderation endpoints

9

Validation & Error Handling

Middleware & global handler

10

Security & Code Quality

Hardening pass

11

Postman Testing

Full collection, success + failure cases

12

README & GitHub

Docs, setup guide, publish



Phases are sequential but not strictly blocking — Security & Code Quality (10) and Postman Testing (11) run continuously alongside every phase from 2 onward, not only at the end.



Final Deliverables

What "done" looks like for the FixMate backend.

- ✓ Complete Node.js backend
- ✓ Express REST APIs
- ✓ MongoDB database
- ✓ JWT authentication
- ✓ Role-based authorization
- ✓ Service management
- ✓ Booking system
- ✓ Provider availability
- ✓ Reviews & ratings
- ✓ Admin APIs
- ✓ Validation
- ✓ Security hardening
- ✓ Postman collection
- ✓ README documentation



GitHub-ready project — clean commit history, a documented setup path, and no secrets checked into version control.



Portfolio Presentation

How to frame this project for a resume, portfolio site, or interview.

PROJECT TITLE

FixMate — Local Service Booking Platform

SHORT DESCRIPTION

A full-stack service booking platform built with Flutter, Node.js, Express.js, and MongoDB that enables customers to discover local services, book providers, track service requests, and submit reviews.

Technologies

Flutter

Dart

Node.js

Express.js

MongoDB

Mongoose

JWT

REST API

Skills to highlight

**Role-Based Authentication****REST API Development****MongoDB Database Design****Booking Workflow****Provider Management****API Security**